



在调用中使用 `std::forward` 保持类型信息

694

我们可以使用一个名为 `forward` 的新标准库设施来传递 `flip2` 的参数, 它能保持原始实参的类型。类似 `move`, `forward` 定义在头文件 `utility` 中。与 `move` 不同, `forward` 必须通过显式模板实参来调用 (参见 16.2.2 节, 第 603 页)。 `forward` 返回该显式实参类型的右值引用。即, `forward<T>` 的返回类型是 `T&&`。

C++
11

通常情况下, 我们使用 `forward` 传递那些定义为模板类型参数的右值引用的函数参数。通过其返回类型上的引用折叠, `forward` 可以保持给定实参的左值/右值属性:

```
template <typename Type> intermediary(Type &&arg)
{
    finalFcn(std::forward<Type>(arg));
    // ...
}
```

本例中我们使用 `Type` 作为 `forward` 的显式模板实参类型, 它是从 `arg` 推断出来的。由于 `arg` 是一个模板类型参数的右值引用, `Type` 将表示传递给 `arg` 的实参的所有类型信息。如果实参是一个右值, 则 `Type` 是一个普通 (非引用) 类型, `forward<Type>` 将返回 `Type&&`。如果实参是一个左值, 则通过引用折叠, `Type` 本身是一个左值引用类型。在此情况下, 返回类型是一个指向左值引用类型的右值引用。再次对 `forward<Type>` 的返回类型进行引用折叠, 将返回一个左值引用类型。



当用于一个指向模板参数类型的右值引用函数参数 (`T&&`) 时, `forward` 会保持实参类型的所有细节。

使用 `forward`, 我们可以再次重写翻转函数:

```
template <typename F, typename T1, typename T2>
void flip(F f, T1 &&t1, T2 &&t2)
{
    f(std::forward<T2>(t2), std::forward<T1>(t1));
}
```

如果我们调用 `flip(g, i, 42)`, `i` 将以 `int&` 类型传递给 `g`, `42` 将以 `int&&` 类型传递给 `g`。



与 `std::move` 相同, 对 `std::forward` 不使用 `using` 声明是一个好主意。我们将在 18.2.3 节 (第 706 页) 中解释原因。

16.2.7 节练习

练习 16.47: 编写你自己版本的翻转函数, 通过调用接受左值和右值引用参数的函数来测试它。



16.3 重载与模板

函数模板可以被另一个模板或一个普通非模板函数重载。与往常一样, 名字相同的函数必须具有不同数量或类型的参数。

695

如果涉及函数模板, 则函数匹配规则 (参见 6.4 节, 第 209 页) 会在以下几方面受到

影响:

- 对于一个调用, 其候选函数包括所有模板实参推断 (参见 16.2 节, 第 600 页) 成功的函数模板实例。
- 候选的函数模板总是可行的, 因为模板实参推断会排除任何不可行的模板。
- 与往常一样, 可行函数 (模板与非模板) 按类型转换 (如果对此调用需要的话) 来排序。当然, 可以用于函数模板调用的类型转换是非常有限的 (参见 16.2.1 节, 第 601 页)。
- 与往常一样, 如果恰有一个函数提供比任何其他函数都更好的匹配, 则选择此函数。但是, 如果有多个函数提供同样好的匹配, 则:
 - 如果同样好的函数中只有一个是非模板函数, 则选择此函数。
 - 如果同样好的函数中没有非模板函数, 而有多于一个函数模板, 且其中一个模板比其他模板更特例化, 则选择此模板。
 - 否则, 此调用有歧义。



正确定义一组重载的函数模板需要对类型间的关系及模板函数允许的有限的实参类型转换有深刻的理解。

编写重载模板

作为一个例子, 我们将构造一组函数, 它们在调试中可能很有用。我们将这些调试函数命名为 `debug_rep`, 每个函数都返回一个给定对象的 `string` 表示。我们首先编写此函数的最通用版本, 将它定义为一个模板, 接受一个 `const` 对象的引用:

```
// 打印任何我们不能处理的类型
template <typename T> string debug_rep(const T &t)
{
    ostringstream ret; // 参见 8.3 节 (第 287 页)
    ret << t; // 使用 T 的输出运算符打印 t 的一个表示形式
    return ret.str(); // 返回 ret 绑定的 string 的一个副本
}
```

此函数可以用来生成一个对象对应的 `string` 表示, 该对象可以是任意具备输出运算符的类型。

696

接下来, 我们将定义打印指针的 `debug_rep` 版本:

```
// 打印指针的值, 后跟指针指向的对象
// 注意: 此函数不能用于 char*; 参见 16.3 节 (第 617 页)
template <typename T> string debug_rep(T *p)
{
    ostringstream ret;
    ret << "pointer: " << p; // 打印指针本身的值
    if (p)
        ret << " " << debug_rep(*p); // 打印 p 指向的值
    else
        ret << " null pointer"; // 或指出 p 为空
    return ret.str(); // 返回 ret 绑定的 string 的一个副本
}
```

此版本生成一个 `string`, 包含指针本身的值和调用 `debug_rep` 获得的指针指向的值。注意此函数不能用于打印字符指针, 因为 `IO` 库为 `char*` 值定义了一个 `<<` 版本。此 `<<` 版本假定指针表示一个空字符结尾的字符数组, 并打印数组的内容而非地址值。我们将在 16.3

节（第 617 页）介绍如何处理字符指针。

我们可以这样使用这些函数：

```
string s("hi");  
cout << debug_rep(s) << endl;
```

对于这个调用，只有第一个版本的 `debug_rep` 是可行的。第二个 `debug_rep` 版本要求一个指针参数，但在此调用中我们传递的是一个非指针对象。因此编译器无法从一个非指针实参实例化一个期望指针类型参数的函数模板，因此实参推断失败。由于只有一个可行函数，所以此函数被调用。

如果我们用一个指针调用 `debug_rep`：

```
cout << debug_rep(&s) << endl;
```

两个函数都生成可行的实例：

- `debug_rep(const string*&)`，由第一个版本的 `debug_rep` 实例化而来，`T` 被绑定到 `string*`。
- `debug_rep(string*)`，由第二个版本的 `debug_rep` 实例化而来，`T` 被绑定到 `string`。

第二个版本的 `debug_rep` 的实例是此调用的精确匹配。第一个版本的实例需要进行普通指针到 `const` 指针的转换。正常函数匹配规则告诉我们应该选择第二个模板，实际上编译器确实选择了这个版本。

697

多个可行模板

作为另外一个例子，考虑下面的调用：

```
const string *sp = &s;  
cout << debug_rep(sp) << endl;
```

此例中的两个模板都是可行的，而且两个都是精确匹配：

- `debug_rep(const string*&)`，由第一个版本的 `debug_rep` 实例化而来，`T` 被绑定到 `string*`。
- `debug_rep(const string*)`，由第二个版本的 `debug_rep` 实例化而来，`T` 被绑定到 `const string`。

在此情况下，正常函数匹配规则无法区分这两个函数。我们可能觉得这个调用将是有歧义的。但是，根据重载函数模板的特殊规则，此调用被解析为 `debug_rep(T*)`，即，更特例化的版本。

设计这条规则的原因是，没有它，将无法对一个 `const` 的指针调用指针版本的 `debug_rep`。问题在于模板 `debug_rep(const T&)` 本质上可以用于任何类型，包括指针类型。此模板比 `debug_rep(T*)` 更通用，后者只能用于指针类型。没有这条规则，传递 `const` 的指针的调用永远是有歧义的。



当有多个重载模板对一个调用提供同样好的匹配时，应选择最特例化的版本。

非模板和模板重载

作为下一个例子，我们将定义一个普通非模板版本的 `debug_rep` 来打印双引号包围

的 string:

```
// 打印双引号包围的 string
string debug_rep(const string &s)
{
    return '"' + s + '"';
}
```

现在, 我们对一个 string 调用 debug_rep 时:

```
string s("hi");
cout << debug_rep(s) << endl;
```

有两个同样好的可行函数:

- debug_rep<string>(const string&), 第一个模板, T 被绑定到 string*。
- debug_rep(const string&), 普通非模板函数。

在本例中, 两个函数具有相同的参数列表, 因此显然两者提供同样好的匹配。但是, 编译器会选择非模板版本。当存在多个同样好的函数模板时, 编译器选择最特例化的版本, 出于相同的原因, 一个非模板函数比一个函数模板更好。

698



对于一个调用, 如果一个非函数模板与一个函数模板提供同样好的匹配, 则选择非模板版本。

重载模板和类型转换

还有一种情况我们到目前为止尚未讨论: C 风格字符串指针和字符串字面常量。现在有了一个接受 string 的 debug_rep 版本, 我们可能期望一个传递字符串的调用会匹配这个版本。但是, 考虑这个调用:

```
cout << debug_rep("hi world!") << endl; // 调用 debug_rep(T*)
```

本例中所有三个 debug_rep 版本都是可行的:

- debug_rep(const T&), T 被绑定到 char[10]。
- debug_rep(T*), T 被绑定到 const char。
- debug_rep(const string&), 要求从 const char* 到 string 的类型转换。

对给定实参来说, 两个模板都提供精确匹配——第二个模板需要进行一次(许可的)数组到指针的转换, 而对于函数匹配来说, 这种转换被认为是精确匹配(参见 6.6.1 节, 第 219 页)。非模板版本是可行的, 但需要进行一次用户定义的类型转换, 因此它没有精确匹配那么好, 所以两个模板成为可能调用的函数。与之前一样, T* 版本更加特例化, 编译器会选择它。

如果我们希望将字符指针按 string 处理, 可以定义另外两个非模板重载版本:

```
// 将字符指针转换为 string, 并调用 string 版本的 debug_rep
string debug_rep(char *p)
{
    return debug_rep(string(p));
}
string debug_rep(const char *p)
{
    return debug_rep(string(p));
}
```

缺少声明可能导致程序行为异常

值得注意的是, 为了使 `char*` 版本的 `debug_rep` 正确工作, 在定义此版本时, `debug_rep(const string&)` 的声明必须在作用域中。否则, 就可能调用错误的 `debug_rep` 版本:

699

```
template <typename T> string debug_rep(const T &t);
template <typename T> string debug_rep(T *p);
// 为了使 debug_rep(char*) 的定义正确工作, 下面的声明必须在作用域中
string debug_rep(const string &);
string debug_rep(char *p)
{
    // 如果接受一个 const string& 的版本的声明不在作用域中,
    // 返回语句将调用 debug_rep(const T&) 的 T 实例化为 string 的版本
    return debug_rep(string(p));
}
```

通常, 如果使用了一个忘记声明的函数, 代码将编译失败。但对于重载函数模板的函数而言, 则不是这样。如果编译器可以从模板实例化出与调用匹配的版本, 则缺少的声明就不重要了。在本例中, 如果忘记了声明接受 `string` 参数的 `debug_rep` 版本, 编译器会默认地实例化接受 `const T&` 的模板版本。



在定义任何函数之前, 记得声明所有重载的函数版本。这样就不必担心编译器由于未遇到你希望调用的函数而实例化一个并非你所需的版本。

16.3 节练习

练习 16.48: 编写你自己版本的 `debug_rep` 函数。

练习 16.49: 解释下面每个调用会发生什么:

```
template <typename T> void f(T);
template <typename T> void f(const T*);
template <typename T> void g(T);
template <typename T> void g(T*);
int i = 42, *p = &i;
const int ci = 0, *p2 = &ci;
g(42); g(p); g(ci); g(p2);
f(42); f(p); f(ci); f(p2);
```

练习 16.50: 定义上一个练习中的函数, 令它们打印一条身份信息。运行该练习中的代码。如果函数调用的行为与你预期不符, 确定你理解了原因。



16.4 可变参数模板

C++
11

一个可变参数模板 (variadic template) 就是一个接受可变数目参数的模板函数或模板类。可变数目的参数被称为参数包 (parameter packet)。存在两种参数包: 模板参数包 (template parameter packet), 表示零个或多个模板参数; 函数参数包 (function parameter packet), 表示零个或多个函数参数。

700

我们用一个省略号来指出一个模板参数或函数参数表示一个包。在一个模板参数列表